

NAME

`remove_stale_templates.py` – quarantine stale manual saved templates and prune their schedlist lines

SYNOPSIS

`remove_stale_templates.py --data-dir DIR --quarantine-dir DIR [--years N] [--execute | --restore RUN_DIR]`

DESCRIPTION

`remove_stale_templates.py` finds *stale saved templates* in a Symphony ILS report-scheduler data directory (*rptsched*) and moves their files into quarantine, removing the corresponding line from *schedlist*.

A saved template (one row of *schedlist*, plus its `<id>.set` / `<id>.selans` file group) is a stale-template candidate when **all** of the following hold:

- **frequency_flag** is **n** – the template is manual (run by hand only), not on a recurring Symphony schedule.
- **owner** does not contain the substring **ACQ**, case-insensitive (the ACQ exclusion – Acquisitions-owned templates are never a candidate, regardless of activity).
- It satisfies the **inactivity rule**: **last_run** is not the "never run" sentinel (**0000000000**) and is at least **--years** years before today; or, if the template has never been run, **created** is at least **--years** years before today.

Unlike orphans, every stale-template candidate has a live *schedlist* line. This tool removes that line – via write-temp-file plus atomic rename, never an in-place edit – so nothing is left dangling. Restoring a run re-inserts the exact removed lines back into the *current* schedlist.

OPTIONS

--data-dir DIR

Path to the *rptsched* data directory to scan and modify. Required in every mode, including **--restore**.

--quarantine-dir DIR

Path to the quarantine directory that holds run subfolders. Created if it does not already exist. Required in every mode.

--years N

Inactivity threshold in years, applied to both **last_run** and, when the template has never run, **created**. Default: 3.

--execute

Perform the move and the *schedlist* rewrite. Without this flag the script runs in dry-run mode and writes nothing.

--restore RUN_DIR

Reverse a previous **--execute** run: restore quarantined files, then re-insert their removed *schedlist* lines. Mutually exclusive with **--execute**.

MODES**Dry run (default)**

Prints the count of stale-template candidates and total files, then one line per candidate: id, description, owner, frequency_flag, created, last_run, and its file list. Nothing on disk is touched.

Execute

With **--execute**, the script:

1. Finds candidates as described above. If none are found, prints a message and exits 0 without creating a run subfolder.
2. Creates **--quarantine-dir/templates_TIMESTAMP** and moves each candidate's file group into it, writing *manifest.csv* (same mechanics and partial-failure behavior as **remove_orphans.py(1)**).

3. Writes the exact removed *schedlist* lines, verbatim and in id order, to *removed_schedlist_lines.txt* in the run subfolder.
4. Rewrites *schedlist* to drop those lines, via a temp file plus atomic rename.

Steps 3 and 4 only run after every file in step 2 has been moved successfully; a move failure aborts before *schedlist* is touched, so a failed run never produces a dangling *schedlist* line.

Restore

With **--restore** *RUN_DIR*, the script:

1. Restores quarantined files exactly as **remove_orphans.py**(1)'s **--restore** does: idempotent, resumable, skips rows whose *dest_path* no longer exists, aborts on a *source_path* collision.
2. Reads *RUN_DIR/removed_schedlist_lines.txt* (if present – an empty result is treated as "no lines to reinsert", not an error) and re-inserts each line into the **current** schedlist, then re-sorts the file by id.

The re-insert step is idempotent independently of the file-restore step: it skips any line whose id is already present in the current schedlist, so it is safe even if the current schedlist has legitimate edits made after the original removal run, and safe to re-run.

EXIT STATUS

- 0** Success – including a clean dry run, an execute run that found zero candidates, or a restore that skipped everything because it was already complete.
- 1** A move, schedlist rewrite, or restore operation failed. Details are printed to stderr.

FILES

<*data-dir*>/*schedlist*

Read to find candidates; rewritten in place (atomically) by **--execute** to drop matched lines, and again by **--restore** to re-insert them.

<*quarantine-dir*>/*templates_<timestamp>/manifest.csv*

File-move record, same format as **remove_orphans.py**(1).

<*quarantine-dir*>/*templates_<timestamp>/removed_schedlist_lines.txt*

The exact *schedlist* lines removed by this run, one per line, verbatim. Never deleted – required for **--restore** and doubles as an audit record.

EXAMPLES

Preview stale templates using the default 3-year threshold:

```
remove_stale_templates.py --data-dir ./rptsched --quarantine-dir ./quarantine
```

Use a 5-year threshold instead:

```
remove_stale_templates.py --data-dir ./rptsched --quarantine-dir ./quarantine --years 5
```

Execute a run:

```
remove_stale_templates.py --data-dir ./rptsched --quarantine-dir ./quarantine --execute
```

Undo a specific run (files and schedlist lines both):

```
remove_stale_templates.py --data-dir ./rptsched --quarantine-dir ./quarantine --restore  
./quarantine/templates_20260826_140501
```

SEE ALSO

remove_orphans.py(1), **sync_operator_field.py**(1)

Domain reference: *rptsched-domain-reference.md* in the rptsched-cleanup repository – see "Inactivity rule" and "ACQ exclusion" under "Definitions", and the *schedlist* field table.

CAVEATS

Requires Python 3.6.8, standard library only.

The **--years** value must match between the run that removed a template and any later reasoning about it; re-store replays the exact removed line rather than re-evaluating staleness, so it is unaffected by a later change to **--years**.

This tool only ever considers manual templates (**frequency_flag** == "n"). Recurring-schedule templates are the separate, not-yet-built "scheduled report removal candidates" category described in the project's CLAUDE.md.